OBJECT-ORIENTED PROGRAMMING (OOP)

A Step-by-Step Learning Guide for CSE Students

Primary language	C++
Level	Beginner to Intermediate
Goal	Understand concepts + write programs + prepare for exams/interviews
Recommended pace	10-14 days, 60-90 minutes per day

How to use this PDF: Learn one section at a time. Type every code example yourself, predict the output before running it, and complete the practice questions at the end of each stage.

Learning path: Programming basics -> Classes & Objects -> Constructors -> Encapsulation -> Inheritance -> Polymorphism -> Abstraction -> Relationships -> Advanced C++ OOP -> SOLID basics -> Mini projects.

Contents

Important: OOP is not just syntax. It is a way to model a problem using objects that contain both *state* (data) and *behavior* (functions).

1. OOP Mindset and Prerequisites

1.1 What is Object-Oriented Programming?

Object-Oriented Programming organizes a program around **objects**. An object represents an entity such as a Student, BankAccount, Car, Employee or GamePlayer. Each object contains data and the operations that work on that data.

Term	Meaning	Example
State	Data stored in an object	Student name, roll number, marks
Behavior	Actions an object can perform	display(), calculateGrade()
Identity	What makes one object distinct	Student ID 101 vs Student ID 102

1.2 Why use OOP?

- **Modularity:** divide a large program into independent classes.
- **Reusability:** reuse existing classes through inheritance/composition.
- **Maintainability:** changes can be isolated inside a class.
- **Security:** hide internal data using encapsulation.
- **Real-world modeling:** map software entities to real concepts.
- **Scalability:** easier to extend medium and large applications.

1.3 Four Core Pillars

Pillar	Simple idea
Encapsulation	Protect data and bundle it with related methods.
Inheritance	Create a new class from an existing class.
Polymorphism	One interface, many behaviors.
Abstraction	Show essential features and hide implementation details.

1.4 Prerequisites to revise

- Variables and data types
- if/else and switch
- loops
- functions
- arrays and strings
- pointers and references (basic level)
- structures (helpful but not compulsory)

Rule: Do not memorize definitions first. Build a small program, then connect the program to the definition.

2. Classes and Objects

2.1 Class

A **class** is a user-defined blueprint or template. It specifies which data and functions an object will have.

```
#include <iostream>
#include <string>
using namespace std;

class Student {
public:
    string name;
    int marks;

    void display() {
        cout << name << " " << marks << endl;
    }
};

int main() {
    Student s1;          // object
    s1.name = "Arun";
    s1.marks = 90;
    s1.display();
}
```

2.2 Object

An object is an instance of a class. If **Student** is the blueprint, **s1** is one actual student object created from that blueprint.

Syntax

```
ClassName objectName;

Student s1;
Student s2;
```

2.3 Member access operator

Use the dot operator **.** for normal objects. Use the arrow operator **->** when accessing an object through a pointer.

```
Student s;
s.display();

Student* p = &s;
p->display();
```

3. Access Specifiers and Encapsulation

3.1 public, private and protected

Specifier	Inside class	Outside class	Main use
public	Yes	Yes	Public interface
private	Yes	No	Internal data/implementation
protected	Yes	No directly	Inheritance

3.2 Encapsulation

Encapsulation means bundling data and methods inside a class and controlling access to the internal state. A common pattern is to keep data **private** and expose validated public methods.

```
class BankAccount {
private:
    double balance;

public:
    BankAccount() : balance(0) {}

    void deposit(double amount) {
        if (amount > 0)
            balance += amount;
    }

    double getBalance() const {
        return balance;
    }
};
```

Outside code cannot directly set **balance = -5000**. It must use the public interface, which can validate input.

3.3 Getters and setters

```
class Student {
private:
    int marks;

public:
    void setMarks(int m) {
        if (m >= 0 && m <= 100)
            marks = m;
    }

    int getMarks() const {
        return marks;
    }
};
```

Exam line: Encapsulation = data hiding + controlled access through class methods.

4. Constructors and Destructors

4.1 Constructor

A constructor is a special member function called automatically when an object is created. It has the same name as the class and has no return type.

```
class Student {
private:
    string name;
    int marks;
```

```
public:
    Student() {
        name = "Unknown";
        marks = 0;
    }

    Student(string n, int m) {
        name = n;
        marks = m;
    }
};
```

4.2 Initialization list

Initialization lists are the preferred way to initialize data members, especially const members, references and base classes.

```
Student(string n, int m) : name(n), marks(m) {}
```

4.3 Destructor

A destructor runs automatically when an object is destroyed. Its name is the class name prefixed with ~. It is used for cleanup.

```
class Demo {
public:
    Demo()  { cout << "Object created\n"; }
    ~Demo() { cout << "Object destroyed\n"; }
};
```

4.4 Constructor types to know

- Default constructor
- Parameterized constructor
- Copy constructor
- Move constructor (advanced C++)

5. this Pointer, Static Members and const Members

5.1 this pointer

this is an implicit pointer to the current object.

```
class Student {
private:
    int marks;
public:
    void setMarks(int marks) {
        this->marks = marks;
    }
};
```

5.2 Static data member

A static data member belongs to the class itself, not to each individual object. All objects share one copy.

```
class Student {
public:
    static int count;

    Student() {
        count++;
    }
};

int Student::count = 0;
```

5.3 Static member function

```
class Math {
public:
    static int square(int x) {
        return x * x;
    }
};

// Math::square(5)
```

5.4 const member function

A const member function promises not to modify the object's normal data members.

```
double getBalance() const {
    return balance;
}
```

6. Inheritance

6.1 Basic idea

Inheritance allows a derived class to acquire members of a base class. It represents an **IS-A** relationship. Example: Student IS-A Person.

```
class Person {
public:
    string name;
    void introduce() {
        cout << "I am " << name << endl;
    }
};

class Student : public Person {
public:
    int rollNo;
};
```

6.2 Types of inheritance

Type	Structure	Example idea
Single	A -> B	Person -> Student
Multilevel	A -> B -> C	Person -> Employee -> Manager
Hierarchical	A -> B and A -> C	Shape -> Circle, Rectangle
Multiple	A + B -> C	Teacher + Researcher -> Professor
Hybrid	Combination	Large designs

6.3 Constructor/destructor order

- Construction: Base constructor runs first, then derived constructor.
- Destruction: Derived destructor runs first, then base destructor.

7. Polymorphism

Polymorphism means **one name/interface can represent multiple behaviors**. C++ supports compile-time and run-time polymorphism.

Type	When decided	Mechanisms
Compile-time (static)	During compilation	Function overloading, operator overloading
Run-time (dynamic)	During execution	Function overriding + virtual functions

7.1 Simple intuition

A function named **draw()** can behave differently for Circle, Rectangle and Triangle objects. The caller can work with a common Shape interface.

8. Function Overloading and Operator Overloading

8.1 Function overloading

Multiple functions may have the same name if their parameter lists differ.

```
class Printer {
public:
    void print(int x) {
        cout << "Integer: " << x << endl;
    }

    void print(string s) {
        cout << "String: " << s << endl;
    }

    void print(int x, int y) {
        cout << x << ", " << y << endl;
    }
};
```

Return type alone cannot distinguish overloaded functions.

8.2 Operator overloading

Operator overloading defines how an operator behaves for a user-defined type.

```
class Complex {
public:
    int real, imag;

    Complex(int r = 0, int i = 0) : real(r), imag(i) {}

    Complex operator+(const Complex& other) const {
        return Complex(real + other.real,
                       imag + other.imag);
    }
};
```

Do not overuse operator overloading. The meaning should remain natural and predictable.

9. Overriding, Virtual Functions and Dynamic Binding

9.1 Function overriding

A derived class provides its own implementation of a base-class function with the same signature.

9.2 Why virtual is needed

```
class Animal {
public:
    virtual void sound() {
        cout << "Animal sound\n";
    }

    virtual ~Animal() = default;
};

class Dog : public Animal {
public:
    void sound() override {
        cout << "Bark\n";
    }
};

int main() {
    Animal* a = new Dog();
    a->sound();     // Bark
    delete a;
}
```

Because **sound()** is virtual, C++ chooses the function using the actual object type at run time. This is dynamic binding.

9.3 virtual destructor

When a base class is meant to be used polymorphically, its destructor should normally be virtual.

10. Abstraction and Abstract Classes

10.1 Abstraction

Abstraction focuses on **what an object can do** rather than exposing **how it does it**. Users interact through a simple public interface.

10.2 Pure virtual function

```
class Shape {
public:
    virtual double area() const = 0;
    virtual ~Shape() = default;
};
```

A class containing at least one pure virtual function is an **abstract class**. You cannot directly create an object of it.

10.3 Derived implementation

```
class Rectangle : public Shape {
private:
    double w, h;

public:
    Rectangle(double w, double h) : w(w), h(h) {}

    double area() const override {
        return w * h;
    }
};
```

10.4 Encapsulation vs abstraction

Encapsulation	Abstraction
Controls access to data/implementation.	Hides unnecessary implementation details.
Often uses private/protected/public.	Often uses interfaces/abstract classes.
Focus: protection and bundling.	Focus: simplified view of complexity.

11. Association, Aggregation and Composition

Not every relationship should use inheritance. Often, one object simply **has** or **uses** another object.

Relationship	Meaning	Lifetime dependency	Example
Association	Objects interact	Independent	Teacher teaches Student
Aggregation	Weak HAS-A	Child can exist independently	Department has Teachers
Composition	Strong HAS-A	Child belongs to owner	Car has Engine

11.1 Composition example

```
class Engine {
public:
    void start() {
        cout << "Engine started\n";
    }
};

class Car {
```

```
private:
    Engine engine;

public:
    void startCar() {
        engine.start();
    }
};
```

Design rule: Prefer composition when the relationship is HAS-A. Use inheritance when the relationship is truly IS-A.

12. Interfaces in C++ and Multiple Inheritance

12.1 Interface-style class

C++ has no separate **interface** keyword. A class containing only pure virtual functions can act as an interface.

```
class Printable {
public:
    virtual void print() const = 0;
    virtual ~Printable() = default;
};
```

12.2 Multiple inheritance

```
class Scanner {
public:
    virtual void scan() = 0;
};

class Printer {
public:
    virtual void print() = 0;
};

class AllInOne : public Scanner, public Printer {
public:
    void scan() override { cout << "Scanning\n"; }
    void print() override { cout << "Printing\n"; }
};
```

12.3 Diamond problem

If two parent classes inherit from the same base and a child inherits from both parents, duplicate base subobjects may appear. C++ can solve this with **virtual inheritance**.

```
class A {};
class B : virtual public A {};
class C : virtual public A {};
class D : public B, public C {};
```

13. Copy Constructor, Shallow Copy and Deep Copy

13.1 Copy constructor

A copy constructor creates a new object from an existing object.

```
class Student {
public:
    string name;

    Student(string n) : name(n) {}

    Student(const Student& other)
        : name(other.name) {}
};
```

13.2 Shallow copy

A shallow copy copies member values directly. If a member is a raw pointer, both objects may point to the same memory.

13.3 Deep copy

A deep copy allocates separate memory and copies the actual pointed-to data.

```
class Box {
private:
    int* value;

public:
    Box(int v) : value(new int(v)) {}

    Box(const Box& other)
        : value(new int(*other.value)) {}

    ~Box() {
        delete value;
    }
};
```

13.4 Rule of Three / Five / Zero

- **Rule of Three:** If you manually define destructor, copy constructor or copy assignment, you often need all three.
- **Rule of Five:** Modern C++ adds move constructor and move assignment.
- **Rule of Zero:** Prefer standard library types and smart pointers so you do not manually manage resources.

14. Memory Management and RAII

14.1 Stack vs heap

Stack object	Heap object
Automatic lifetime	Dynamic lifetime
Student s;	Student* p = new Student;
Destroyed automatically at scope end	Must be released unless a smart pointer manages it

14.2 Prefer smart pointers

In modern C++, prefer **std::unique_ptr** or **std::shared_ptr** instead of raw new/delete when dynamic ownership is necessary.

```
#include <memory>

auto s = std::make_unique<Student>();
// automatically destroyed when ownership ends
```

14.3 RAII

RAII (Resource Acquisition Is Initialization) means tie a resource's lifetime to an object's lifetime. Constructors acquire resources and destructors release them.

15. Exception Handling in OOP

Exceptions allow an operation to report an abnormal condition without returning special error values everywhere.

```
class BankAccount {
private:
    double balance = 0;

public:
    void withdraw(double amount) {
        if (amount > balance)
            throw runtime_error("Insufficient balance");
        balance -= amount;
    }
};
```

```
try {
    account.withdraw(5000);
}
catch (const exception& e) {
    cout << e.what();
}
```

Good practice

- Throw exceptions for exceptional situations, not normal control flow.
- Catch by const reference.
- Use RAII so resources are cleaned automatically if an exception occurs.

16. Templates and Generic Programming

Templates allow one piece of code to work with many data types. This is not one of the four OOP pillars, but it is essential modern C++.

```
template <typename T>
class Box {
private:
    T value;

public:
    Box(T v) : value(v) {}

    T getValue() const {
        return value;
    }
};

Box<int> a(10);
Box<string> b("Hello");
```

OOP vs generic programming

OOP often varies behavior through inheritance and virtual functions. Templates vary behavior/types at compile time. Real C++ programs commonly use both.

17. SOLID Principles - Beginner Introduction

SOLID is a set of design guidelines that helps classes remain easier to change and test. Learn the ideas after you understand the four OOP pillars.

Letter	Principle	Beginner meaning
S	Single Responsibility	A class should have one main reason to change.
O	Open/Closed	Open for extension, closed for unnecessary modification.
L	Liskov Substitution	A derived object should safely replace its base object.
I	Interface Segregation	Prefer small focused interfaces.
D	Dependency Inversion	Depend on abstractions, not concrete details.

Example: Single Responsibility

A Student class should represent student data/behavior. It should not also handle database connection, PDF printing, email sending and payment processing. Split unrelated responsibilities into separate classes.

18. Common Mistakes and Debugging Checklist

Mistake	Better approach
Making all members public	Keep state private; expose meaningful methods.
Using inheritance for every relationship	Check IS-A vs HAS-A first.
Forgetting virtual destructor in polymorphic base	Use virtual ~Base() = default;
Using raw new/delete everywhere	Prefer automatic objects and smart pointers.
Confusing overloading and overriding	Overloading: different params. Overriding: derived replaces virtual base function.
Writing huge classes	Split responsibilities.

Debugging questions

- Which object owns this data/resource?
- Which constructor is being called?
- Is the method const-correct?
- Should this base method be virtual?
- Is this relationship IS-A or HAS-A?
- Could two objects be deleting the same pointer?

19. Interview and Exam Revision Sheet

Question	Short answer
What is a class?	A blueprint defining data and member functions.
What is an object?	An instance of a class.
What is encapsulation?	Bundling data/methods and controlling access.
What is inheritance?	Deriving a class from an existing class to reuse/extend behavior.
What is polymorphism?	One interface with multiple implementations/behaviors.
What is abstraction?	Expose essentials; hide implementation details.
Overloading vs overriding?	Overloading: different parameter lists. Overriding: derived redefines virtual base method.
Compile-time vs run-time polymorphism?	Overloading/operators vs virtual functions/overriding.
What is a constructor?	Special function automatically called during object creation.
What is a destructor?	Special function automatically called during destruction.
What is a pure virtual function?	A virtual function declared with = 0.
What is an abstract class?	A class with at least one pure virtual function.
What is deep copy?	Create independent copies of dynamically owned data.
What is RAII?	Bind resource lifetime to object lifetime.

Fast comparison

Concept A	Concept B	Key difference
Class	Object	Blueprint vs instance
Encapsulation	Abstraction	Control access vs hide complexity
Inheritance	Composition	IS-A vs HAS-A
Overloading	Overriding	Compile-time vs normally run-time
Static binding	Dynamic binding	Compile-time selection vs run-time selection

20. 12-Day Step-by-Step Study Plan

Day	Learn	Practice
Day 1	OOP idea, class, object, methods	Create Student and Car classes
Day 2	private/public, getters/setters, encapsulation	BankAccount with validation
Day 3	Constructors, destructors, this, const	Employee objects
Day 4	Static members + revision	Count created objects
Day 5	Inheritance basics	Person -> Student
Day 6	Inheritance types + composition	Vehicle hierarchy
Day 7	Function/operator overloading	Calculator + Complex

Day	Learn	Practice
Day 8	Overriding + virtual functions	Animal sounds
Day 9	Abstract class + pure virtual functions	Shape area system
Day 10	Copying, pointers, RAII	Deep-copy practice
Day 11	SOLID basics + design questions	Refactor one large class
Day 12	Mini project + full revision	Library or banking system

21. Practice Programs

Beginner

- Create a Student class with name, roll number and marks; print grade.
- Create a Rectangle class with length and breadth; find area/perimeter.
- Create a BankAccount class with private balance, deposit and withdraw.
- Create a Product class with constructor and discountedPrice().
- Use a static member to count how many objects have been created.

Intermediate

- Create Person -> Student and Person -> Teacher using inheritance.
- Create Shape as an abstract class; derive Circle and Rectangle.
- Create Animal base class and Dog/Cat derived classes using virtual sound().
- Overload + for Complex numbers.
- Build a Library system using Book and Member classes with composition/association.

Mini projects

Project	Classes you may use	OOP concepts
Library Management	Book, Member, Library, Transaction	Encapsulation, association/composition
Banking System	Account, SavingsAccount, CurrentAccount	Inheritance, polymorphism
Student Result System	Student, Subject, Result	Encapsulation, composition
Parking System	Vehicle, Car, Bike, ParkingSlot	Inheritance, abstraction
Simple Shopping Cart	Product, CartItem, Cart, Customer	Composition, constructors

22. Capstone Example: Shape Polymorphism

This program combines abstraction, inheritance, overriding, virtual functions, dynamic binding and smart pointers.

```
#include <iostream>
#include <vector>
#include <memory>
using namespace std;

class Shape {
public:
    virtual double area() const = 0;
    virtual void display() const = 0;
    virtual ~Shape() = default;
};

class Rectangle : public Shape {
private:
    double w, h;

public:
    Rectangle(double w, double h) : w(w), h(h) {}

    double area() const override {
        return w * h;
    }

    void display() const override {
        cout << "Rectangle area = " << area() << endl;
    }
};

class Circle : public Shape {
private:
    double r;

public:
    Circle(double r) : r(r) {}

    double area() const override {
        return 3.14159 * r * r;
    }

    void display() const override {
        cout << "Circle area = " << area() << endl;
    }
};

int main() {
    vector<unique_ptr<Shape>> shapes;

    shapes.push_back(make_unique<Rectangle>(4, 5));
    shapes.push_back(make_unique<Circle>(3));

    for (const auto& s : shapes)
        s->display();
}
```

What concepts are present?

- **Abstraction:** Shape declares what derived shapes must provide.
- **Inheritance:** Rectangle and Circle inherit Shape.
- **Polymorphism:** One vector stores different kinds of shapes through Shape pointers.
- **Overriding:** Each derived class implements area() and display().
- **Dynamic binding:** The correct display() is selected at run time.

- **Encapsulation:** dimensions are private.
- **RAII:** unique_ptr manages memory automatically.

23. Self-Test

- 1. Why is an object called an instance of a class?
- 2. Why should data members usually be private?
- 3. What is the difference between a default and parameterized constructor?
- 4. In what order do base and derived constructors run?
- 5. Give one example of compile-time polymorphism.
- 6. Why is virtual needed for run-time polymorphism?
- 7. What does override help the compiler check?
- 8. Can an abstract class have normal member functions?
- 9. When should composition be preferred over inheritance?
- 10. What problem can shallow copy cause with raw pointers?
- 11. What is RAII?
- 12. Explain the four pillars using one example each.

Target: If you can answer all 12 without looking back, explain the capstone program line-by-line, and complete at least two mini projects, you have a strong foundation in OOP.

24. Final Learning Advice

- Type code yourself instead of only reading it.
- Draw small class diagrams on paper: class name, data, functions and relationships.
- After each concept, create one example from daily life and one program.
- Use a debugger to watch constructor/destructor calls and virtual dispatch.
- Do not jump immediately to design patterns; first become comfortable with classes, ownership, inheritance and polymorphism.
- After this guide, learn STL containers, smart pointers, exception safety, UML basics and common design patterns.

One-line memory formula: Class = blueprint, Object = instance, Encapsulation = protect, Inheritance = reuse/extend, Polymorphism = many behaviors, Abstraction = hide complexity.